

The Stack, and Why It Isn't MERN

What we're running, why each piece is there, and a point-by-point case against MERN's architectural commitments — including the places where MERN still wins. Written against the codebase as it stands: **Next.js 16.2**, **React 19.2**, **~4,000 lines of TypeScript across 48 source files**, published at github.com/Aaditeya8/wedding-mvp.

48
SOURCE FILES

6
CLIENT
COMPONENTS

1
DEPLOYABLE APP

0
REST ENDPOINTS

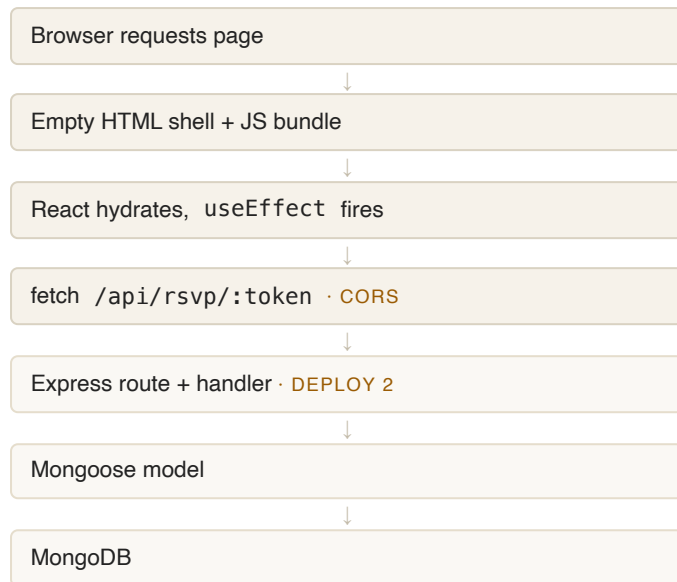
₹0
LOCAL DEV COST

1 · What we're actually running

LAYER	CHOICE	VERSION
Framework	Next.js, App Router — React Server Components + Server Actions	16.2.10
UI runtime	React	19.2.4
Language	TypeScript, strict	5.x
Styling	Tailwind CSS v4 (PostCSS plugin) over CSS custom-property theme tokens	4.x
Data access	Drizzle ORM + drizzle-kit migrations	0.45 / 0.31
Database	Postgres — PGlite embedded locally, Neon serverless in production	0.5 / 1.1
Auth	Auth.js v5 (next-auth), Drizzle adapter, Nodemailer magic links, DB sessions	5.0-beta
Validation	Zod	4.x
Email	React Email for templates, Nodemailer for transport	1.0 / 7.0
Tests	Vitest (unit), Playwright (e2e)	4.1 / 1.61
Host	Vercel	—

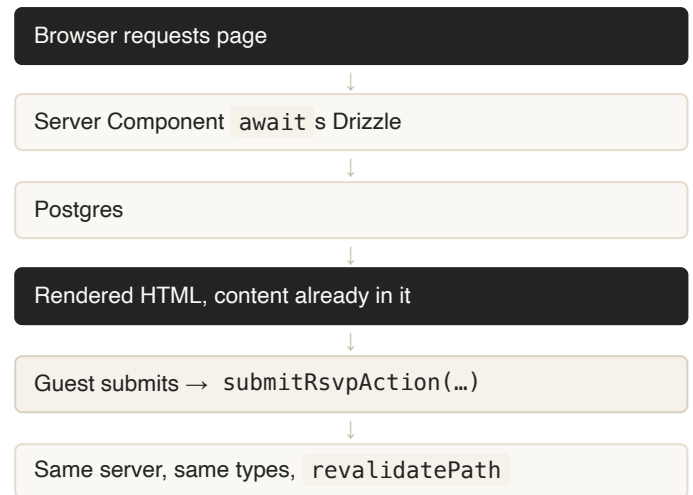
There is no separate backend. No Express app, no `/api` layer beyond the single `Auth.js` catch-all route, no REST client, no Redux or React Query. **The server is the framework.**

A GUEST RSVPS — THE MERN PATH



7 hops · 2 deploys · types re-declared 3 times · guest waits through a spinner

A GUEST RSVPS — HERE



1 deploy · one type graph · the first byte carries the invitation

2 · Why each piece is here

Next.js App Router — because the API layer was pure overhead

The traditional shape of this app: React SPA → `fetch('/api/rsvp')` → Express route → Mongoose model → Mongo. Four places to change when a field is added, three places for a type to drift, two things to deploy. React Server Components collapse the first three. A page reads its own data:

```
// src/app/w/[slug]/page.tsx
export default async function WeddingPage({ params }) {
  const data = await getWeddingBySlug(await params.slug);
  if (!data) notFound();
  ...
}
```

No endpoint, no fetch, no loading state, no serialization contract. Writes go the same way, through Server Actions:

```
// src/app/rsvp/actions.ts
"use server";
export async function submitRsvpAction(input) {
  const ip = (await headers()).get("x-forwarded-for")?.split(",")[0] ?? "local";
  return submitRsvp(input, ip);
}
```

The client component calls `submitRsvpAction(...)` like a local function. Next.js generates the transport. The argument and return types are checked by `tsc` across the network boundary — something no hand-written REST endpoint gives you for free.

The bundle consequence, which is the actual business reason

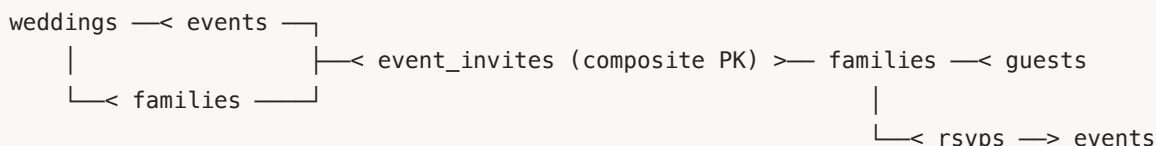
6 of the 48 files in `src/` are client components. Everything else — every dashboard page, every wedding-site section, all the data access — ships *zero JavaScript* to the browser.

That matters because of who the users are. Guests open the RSVP link from a WhatsApp message, on a mid-range Android phone, on Indian mobile data, often at a venue with bad signal. A client-rendered SPA hands them a blank shell, a spinner and a waterfall of API calls. Server-rendered HTML hands them the invitation.

It also matters for the wedding site itself, which exists to be shared. `generateMetadata` runs on the server per slug, so the WhatsApp and Instagram link preview shows the couple's names and date. An SPA renders that after JavaScript executes, which link-preview crawlers do not wait for.

Postgres + Drizzle — because this data is relational, not documents

The domain is a graph of hard relationships:



`src/db/schema.ts` encodes that as real constraints, not conventions:

- **Foreign keys with `onDelete: "cascade"`** — deleting a wedding removes its events, families, guests, invites and RSVPs in a single statement. The seed depends on this to stay idempotent.
- **`uniqueIndex("rsvp_event_family").on(eventId, familyId)`** — one RSVP per family per event, enforced by the database. That single line is what makes the edit-on-revisit flow safe:

```
await db.insert(rsvps).values({...})
    .onConflictDoUpdate({ target: [rsvps.eventId, rsvps.familyId], set: {...} });
```

Double-tap submit on a flaky connection and you get one row, atomically, with no read-modify-write race. In a document store this is application logic you write, test, and eventually get wrong under concurrency.

- **Postgres enums** for theme, side, role, RSVP status and age group — an invalid value is rejected by the database, not by a validator someone forgot to call.
- **Composite primary key** on `event_invites`, the many-to-many between events and families. Which family is invited to which ceremony is the core of the product — a cousin gets the Sangeet, the office gang gets the Reception only. That's a join table.

Drizzle sits on top as a thin, typed SQL builder rather than an abstraction layer. Row types are *inferred from the schema*, so renaming `hero_tagline` breaks the build in the admin dashboard rather than at 11pm during a demo.

PGLite locally, Neon in production — the ₹0 dev environment

`src/db/client.ts` picks the driver from a single environment variable:

```
const url = process.env.DATABASE_URL;
if (url) return drizzleNeon(neon(url), { schema }); // production
const pglite = new PGLite(ephemeral ? undefined : "var/pglite"); // local
return drizzlePglite(pglite, { schema });
```

Locally the entire database is a folder. No Docker, no Postgres install, no cloud account, no cost. `npm run db:reset` is `rm -rf var/pglite && npm run seed`. Tests and production builds get a throwaway **in-memory** instance, so `npm test` can't corrupt dev data and suites don't need cleanup fixtures.

And it is the *same dialect* both sides. Migrations, enums, `ON CONFLICT`, cascades — all verified locally exactly as they'll run on Neon. Compare with the usual SQLite-locally / Postgres-in-prod split, where a migration passes

locally and fails in production.

The trade-off is real and documented: on-disk PGLite is single-process, so seeding while the dev server runs corrupts it. That's the price of a zero-dependency dev database, and it's why `playwright.config.ts` polls `/favicon.ico` rather than a DB-backed route.

Auth.js magic links — because the actual requirement was "no passwords"

Two different access models, both password-free.

Guests never authenticate at all. They get a capability URL: 32 random bytes, base64url. The server stores only `sha256(token)` (`src/lib/tokens.ts`), so a database leak yields no working invite links. Tokens expire seven days after the wedding.

Staff get magic links via Auth.js with database sessions and **no self-signup** — the `signIn` callback rejects any address that isn't already a provisioned user row:

```
// src/auth.ts
async signIn({ user }) { return isProvisionedStaff(user.email); }
```

Nobody hires a wedding committee member who then has to remember a password. Nobody wants a stranger creating an account on the couple's dashboard.

Authorization is layered deliberately. `src/proxy.ts` (Next 16's renamed middleware) only checks *"is there a session cookie?"* and bounces to `/signin` — it never imports the auth module, because database sessions can't be validated in that runtime. The real check, `requireRole()`, runs inside every protected page and action, where the session is a database read:

```
if (!u?.role) redirect("/signin");
if (!allowed.includes(u.role)) redirect(ROLE_HOME[u.role]);
```

Why that's safe: there is no public API surface to bypass. Server Actions execute server-side by definition; there is no `/api/rsvp` a determined guest can `curl` around the check. In a MERN app, every Express route is a door that must be independently guarded.

Tailwind v4 + CSS custom properties — six themes without six stylesheets

The theme system is the product's demo moment: the couple clicks a swatch and their entire site changes. Implementation is one attribute:

```
<main data-theme={wedding.theme} className="themed">
```

`src/themes/tokens.css` (93 lines) defines each theme as a block of custom properties — palette, display face, ornament colours. Tailwind utilities consume the variables, so switching themes re-renders zero components and downloads zero extra CSS. `src/themes/catalog.ts` is the single source of truth that the picker, the Zod schema, the email templates and `scripts/set-theme.ts` all read from. **Six visual identities, ~420 lines of CSS total.**

3 · Why this beats MERN principles

"MERN" is a set of architectural commitments, not just four logos. Taken one at a time.

3.1 · "The frontend and backend are separate applications"

MERN React SPA, Express API, two deploys, CORS, a shared-types package that goes stale, auth verified in two places.

HERE One repository, one build, one deploy, one language, one type graph. A server action is a function call, not an HTTP contract. Adding a field to the RSVP form touches `schema.ts`, the Zod schema and the form — and `tsc` names every other file that needs to change.

For a four-person team where one person is doing all of this alongside a full-time job, eliminating an entire tier isn't an aesthetic preference. It's the difference between shipping the pilot and not.

3.2 · "Render on the client, fetch state over the wire"

MERN Blank HTML, then bundle, then hydrate, then `useEffect`, then fetch, then spinner, then content. State lives in Redux or React Query and has to be invalidated by hand. Link previews and SEO need a separate SSR bolt-on or a prerender service.

HERE HTML arrives with the content in it. `await` in a Server Component is the data layer. There is no cache to invalidate because there is no client cache — `revalidatePath` after a mutation re-renders on the server. No Redux, no React Query, and zero `useEffect` data fetching anywhere in the codebase.

3.3 · "Schemaless is faster to move in"

It's faster for the first week and slower every week after.

GUARANTEE	MONGO + MONGOOSE	POSTGRES + DRIZZLE
One RSVP per family per event	app-level check, racy under concurrency	<code>uniqueIndex</code> + <code>onConflictDoUpdate</code> , atomic
Deleting a wedding cleans up its tree	manual cascade code, or orphans	<code>onDelete: "cascade"</code> , one statement
<code>theme</code> is one of six values	Mongoose enum, in-app only	Postgres enum, enforced in the DB
Guest belongs to a real family	nothing stops a dangling <code>familyId</code>	foreign key
"Invited vs responded, per event"	<code>\$lookup</code> aggregation pipeline	a <code>count(*)</code> and a <code>sum(case when ...)</code>

Mongoose schemas are validation that lives in your application. Postgres constraints are guarantees that live in your data. When the wedding is in three days and the committee is editing families on a phone, the difference is which bugs are *possible*.

The honest counterpoint: if this data really were unstructured — arbitrary per-couple custom fields, event logs, scraped documents — the column would flip. `jsonb` covers most of that case anyway, in the same database, with transactions.

3.4 · "The API is the contract"

MERN A REST endpoint's request and response are `any` on both ends unless you invest in OpenAPI or tRPC. Type drift between the Mongoose model, the Express handler and the React prop is the single most common

source of MERN bugs.

HERE `schema.ts` is the only source of truth. Drizzle infers row types from it, Zod validates untrusted input at the boundary (`submitSchema` in `src/lib/rsvp.ts`), and Server Actions carry the types across the wire. Renaming a column is a compile error, not a runtime `undefined`.

3.5 • "Node + npm gives you everything"

True — and this stack is still Node. That's the part of MERN worth keeping. What it drops is Express, which in 2026 mostly exists to reimplement routing, body parsing and serialization that the framework already does.

3.6 • The scoreboard for *this* product

REQUIREMENT	MERN ANSWER	THIS STACK
Guest RSVPs on 3G at a venue	SPA bundle + API waterfall	server-rendered HTML, ~zero JS on that route
WhatsApp link preview for the site	prerender service or SSR bolt-on	<code>generateMetadata</code> per slug, built in
Multi-tenant weddings, per-role access	auth logic duplicated in Express and React	<code>requireRole()</code> at every entry point, no public API to bypass
No-password sign-in	Passport + custom token table	Auth.js provider, ~40 lines
₹0 local dev	Mongo Atlas free tier or Docker	PGLite — a folder
Demo with no internet	needs Atlas	fully offline, mail to <code>var/outbox/mail.jsonl</code>
One developer, evenings and weekends	two apps to maintain	one

4 • Where MERN — or something else — would win

Stating this plainly, because a comparison that only goes one way isn't an argument.

- **Persistent realtime.** Long-lived WebSockets — live seating charts, a chat channel — fit a stateful Express + Socket.IO server better than serverless functions. If a live feature becomes core, expect to add a small dedicated service.
- **Genuinely schemaless data at scale.** Analytics events, arbitrary ingested documents. Postgres `jsonb` handles the mid-range, but not everything.
- **Hiring.** MERN is the default Indian bootcamp stack. The pool of developers who can productively edit a Server Actions codebase on day one is smaller today.
- **Framework churn.** The sharpest one. Next.js moves fast enough that this repo already ships `src/proxy.ts`, Next 16's rename of `middleware.ts`, and `AGENTS.md` literally warns that the installed version differs from what most documentation describes. `next-auth` is on a `5.0.0-beta`. Express hasn't meaningfully changed in a decade — that stability is worth something.
- **Portability.** Server Components, Server Actions and the Neon driver are portable in principle but comfortable on Vercel. Moving off is real work; an Express app runs anywhere. The mitigation here is that the *domain* logic (`src/lib/*`) is plain TypeScript over Drizzle with no framework imports — that part would move.

The trade: for a pilot that has to look expensive, load fast on bad phones, keep guest data correct, cost nothing to run before the first invoice, and be maintained by one person — it's worth it.

5 • One-paragraph version

We're on Next.js 16 with React Server Components and Server Actions over Postgres via Drizzle, with Auth.js magic links and Tailwind v4 theme tokens. It replaces MERN's SPA-plus-Express-API split with a single deployable app where pages read the database directly and mutations are typed function calls — which means no REST layer, no client state library, and almost no JavaScript shipped to the guests who matter most. We chose Postgres over Mongo because wedding data is a graph of hard relationships and the database, not the application, should be the thing enforcing "one RSVP per family per event." And we run embedded Postgres locally, so the whole product — sign-in and invite emails included — demos on a laptop with no internet and no monthly bill.

Wedding MVP · [docs/framework.html](#) → [docs/Framework.pdf](#) · Markdown source at [docs/Framework.md](#) · 16 August 2026.
File and line references were checked against the published tree at [Aaditeya8/wedding-mvp@main](#).